Orientation, Git, and how mobile apps are built

Course organization, Git and GitHub, app development strategies, Flutter setup

Dinu-Ștefan Rusu ADMD

What you should leave with

Course organization

The fourteen lectures, the seven labs, how the grade is built, and what to bring to Lab 1.

Git setup

A GitHub account and the loop every team runs: branch, commit, push, pull request, review, merge.

Four build strategies

Native, web, cross-platform in two flavors, and shared logic with native user interfaces.

Choosing a strategy

Why this course uses Flutter, what that choice costs, and when another strategy fits better.

1

Course organization

Instructors, lectures, labs, grading

Contents Page

1 **Course organization** 3
2 Git workflow 10
3 App development strategies 18
4 Flutter setup 26

Instructors

Andrei Vasilățeanu

Dinu-Ștefan Rusu
dinu_stefan.rusu@upb.ro

Microsoft Teams: course channel
rusudinu.com

50+
apps shipped

500k+
installs across them

Ask questions during the lecture.

Course scope and the follow-up course

This course

Application Development for Mobile Devices. How to build the app: Dart and Flutter, state, networking, Firebase, authentication, release. Fourteen lectures and seven labs.

The follow-up course

Mobile and Embedded Computing. What runs under the app: runtimes, concurrency, rendering cost, synchronization, remote procedure calls, and the embedded side.

Topics that belong to the second course are only named here.

Lecture schedule

Wk	Lecture
01	Orientation, Git, build strategies
02	Dart, null safety, the toolchain
03	Widgets and layout
04	Async Dart, agent-assisted coding
05	Debugging and state, part 1
06	State management, part 2
07	Networking: HTTP and JSON

Wk	Lecture
08	Firebase, REST and GraphQL
09	Observability and local storage
10	Authentication, OAuth, App Check
11	Routing, permissions, WebSockets
12	AI in the app, LLMs and RAG
13	User interface polish and testing
14	Release, distribution, push

Two hours every week. Each lecture builds on the one before it.

Lab schedule

Week	Lab	What you build
2	Lab 1	Set-up and your first app on a device
4	Lab 2	Dart classes, null-safe types, collections
6	Lab 3	The todo screen, widgets, and DevTools
8	Lab 4	JSON models and a network call with retry
10	Lab 5	The same app, rebuilt on BLoC
12	Lab 6	Authentication, routing, and a live screen
14	Lab 7	The practical lab test

Labs run in even weeks. There is no semester project: the labs grow one app.

Grading

20 p
colloquium

20 p
midterm exam

30 p
lab assignments

30 p
practical lab test in Lab 7, individual

You pass with at least 50 points out of 100.

Lab work and presentation

One app

Every lab continues the same Flutter project. Bring the working app from the previous lab.

Present at the end

At the end of each lab you run your work for the lab instructor and walk through each task.

Explain your code

You answer questions about any line of it.

Every lab is graded live, at the end of the lab. A lab can only be presented in its own session.

2

Git workflow

Git hosting, branches, commits, pull requests, continuous integration

Contents Page

1 Course organization 3
2 Git workflow 10
3 App development strategies 18
4 Flutter setup 26

Git and Git hosting services

- Git is a version control system. It records every change ever made to your source code, and you can return to any of them.
- Several people can work on the same codebase without overwriting each other.
- Git runs on your machine. GitHub, GitLab and Bitbucket are services that host repositories and add collaboration on top: reviews, issues, automation.

One Git, many hosts

The commands are identical everywhere. Only the website around them differs.

This course uses GitHub.

GitHub account

- Sign up, free, at github.com/signup. Use a name you are happy to show a future employer.
- Keep your lab app in a repository of your own. The app grows all semester, and you will want every working state back.
- Turn on two-factor authentication. GitHub requires it for accounts that contribute code.

Shared course material
github.com/rusudinu/presentations

Your own repository
One for your lab app, created in Lab 1

Repositories and branches

- A repository holds all the files of a project and every revision they have ever had.
- It always has at least one branch. By convention that branch is called `main`.
- A branch is a line of work: build a feature or fix a bug without touching anyone else's work.
- A branch is always created from an existing branch, and later merged back into it.

For example

`main` is the working state of your app. `todo-screen` is the screen you are still building.

A repository is a Git concept, not a GitHub one. GitHub only hosts it.

Git workflow commands

```
git clone <repo-url>   # once
git checkout -b todo-screen

# edit files, then:
git add .
git commit -m "Add todo screen"
git push -u origin todo-screen

# open a pull request, get a review

git checkout main
git pull
```

The loop every team runs.
Clone, branch, commit, push, review, merge, pull.

`git status` answers most questions. Run it whenever you are unsure what Git is about to do.

Commit messages

```
Add swipe to delete on the todo list
Fix crash when the title is empty
Move the API base URL into config

fix
update stuff
asdf
```

The first three are useful. They say what changed, in the imperative mood, in one line a reviewer can scan.

Keep the first line short, around seventy characters. One logical change per commit. If you need to explain why, put it in the body under a blank line.

Write the message for the person reading it in six months.

Pull requests and review

- A pull request proposes merging your branch into `main`, and shows the exact diff.
- A reviewer reads it and either approves it or requests changes. Until the required approvals arrive, the pull request stays open.
- You answer comments by pushing more commits to the same branch. The pull request updates itself.

In your labs

The review happens live. You present the work at the end of the lab and answer questions on it.

Open the pull request early, even when the work is unfinished.

Continuous integration and delivery

1 · Test

Continuous integration checks out the merged code, installs dependencies, runs the analyzer and the test suite.

2 · Build

If the tests pass, it builds the release artifacts: an Android bundle, an iOS archive, or a web bundle.

3 · Deliver

The artifact goes to testers or to a store track. Nobody uploads a build from a laptop.

Continuous integration, usually shortened to CI, is a service that runs those steps on a clean machine for every pull request and every merge.

A clean machine shows whether the repository holds everything the project needs.

3

App development strategies

Native, web, cross-platform, shared logic

Contents Page

1 Course organization 3
2 Git workflow 10
3 App development strategies 18
4 Flutter setup 26

Native apps

- Built for one operating system, with that platform's own software development kit and tools.
- Best possible performance, and access to every capability the platform exposes on day one.
- The platform's own widgets, so the app looks and behaves the way the operating system does.

iOS
Swift and SwiftUI, built in Xcode, on a Mac

Android
Kotlin and Jetpack Compose, built in Android Studio

Every other strategy sits on top of these two kits. Cross-platform frameworks do not replace them: they generate or drive the same native application.

Strategy 1: web apps and progressive web apps

What you get

- Opens from a link. No installation and no store review.
- An update reaches every user the moment you deploy it.
- One codebase for every platform that has a browser.

What you give up

- Limited access to sensors and radios: Bluetooth, near-field communication, background location.
- No reliable background execution, especially on iOS.
- A browser performance ceiling, and a single engine on iOS.

A progressive web app, or PWA, is a web app that can be installed to the home screen, works offline, and is served over HTTPS.

Strategy 2: React Native

Your code
JavaScript or TypeScript, running in a JavaScript engine on the device

The layer under it
The JavaScript Interface, JSI, and the Fabric renderer, which call into the platform directly

What the user sees
Real native widgets, one set per platform

- The user interface is native, so each platform keeps its own look and behavior.
- JavaScript can be updated over the air, without a new store release.
- Every interaction crosses the boundary between JavaScript and native code.

The old serialized bridge is gone. Current React Native calls into native code directly, so older descriptions of a JavaScript bridge no longer apply.

Strategy 2: Flutter

Your code
Dart, compiled ahead of time to native machine code for release builds

The layer under it
Impeller, Flutter's renderer, which draws the whole user interface itself

What the user sees
One user interface, identical everywhere

- One rendering pipeline, so there are no per-platform layout surprises.
- It reaches the operating system through platform channels and plugins.
- Hot reload during development: a change appears in the running app in a second.

React Native borrows the platform's widgets. Flutter brings its own.

Strategy 3: Kotlin Multiplatform

Shared Kotlin module
Business logic, networking, storage, models. Compiled natively for each platform.

Android app
Its own user interface, in Jetpack Compose

iOS app
Its own user interface, in SwiftUI or in Compose Multiplatform

- You choose how much to share, from one networking class to everything but the screens.
- The `expect` and `actual` keywords bridge the parts that must differ per platform.
- It is stable and in production at companies such as McDonald's, Netflix and Forbes.

Flutter shares everything, including the pixels. Kotlin Multiplatform shares exactly as much as you choose, and keeps each platform's native screens.

Strategy comparison

Strategy	Shared code	The user interface	Choose it when
Native	none	the platform's own widgets	one platform, every capability
Web and PWA	all of it	HTML and CSS in a browser	reach without an install
React Native	logic and layout	native widgets	the team already writes React
Flutter	everything, drawing included	drawn by Flutter itself	one team, identical screens
Kotlin Multiplatform	as much as you choose	native, written twice	native apps already exist

Choosing a strategy

If this describes you	Start with
You need reach without installs, and light hardware needs	Web or PWA
Your team lives in JavaScript and wants a native feel	React Native
You already ship two native apps and duplicate the logic	Kotlin Multiplatform
One small team, identical screens, fast iteration	Flutter

Team skills, deadline and hardware access usually pick the strategy.

4

Flutter setup

Reasons for Flutter, trade-offs, installation

Contents Page

1 Course organization 3
2 Git workflow 10
3 App development strategies 18
4 Flutter setup 26

Reasons for choosing Flutter

One codebase

One student, working alone, can build a complete app for Android and iOS inside a two-hour lab.

Hot reload

A change shows up in the running app almost immediately, which is what makes the labs work.

One renderer

What you see on the emulator is what a user sees on a phone. Less time lost to platform differences.

It is what I ship

The examples, the packages and the debugging habits come from apps that are in production now.

Flutter trade-offs

- Binaries are larger than a native app of the same size, because the framework travels with the app.
- Talking to the operating system goes through platform channels or a plugin, which is extra work and extra latency.
- Dart is used mostly with Flutter, so the language itself transfers less than Kotlin or TypeScript would.

Say the cost out loud

Every strategy costs something.

Required software

Flutter SDK

The software development kit, from flutter.dev. Add it to your PATH so `flutter` works in any terminal.

Android Studio

Needed for the Android toolchain and the Android emulator, even if you write code in another editor.

Xcode, on a Mac

Only for building and running on iOS and the iOS Simulator. Skip it on Windows or Linux.

Git and an account

Git on your machine, and the GitHub account from Part 2. Both are needed in the first lab.

Install on the machine you bring to the lab. The downloads are large, so start them early.

Installation check and course resources

```
flutter doctor
flutter create hello_admd
cd hello_admd
flutter run
```

Course material
github.com/rusudinu/presentations

Questions
Microsoft Teams course channel, or in the lab

`flutter doctor` prints one line per toolchain component. Every line must be a check mark before Lab 1 starts.

Summary

1. **Every lab ends with a presentation.** One app that grows, shown running to the lab instructor at the end of each lab.
2. **There are four ways to build a mobile app.** Native, web, cross-platform, and shared logic with native screens.
3. **Every strategy has a cost.** This course uses Flutter, and states what that choice gives up.

Further reading

docs.flutter.dev/get-started/install · docs.github.com/get-started · kotlinlang.org multiplatform · reactnative.dev architecture

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel